

T.C.

Veri Yapıları ve Algoritmalar Dersi

HTML'den DOM Ağacı Oluřturma

Proje Raporu

Ad Soyad	Öğrenci Numarası
Mehmet Fatih Erduran	032490002
Kerem Bilgiç	032490001
Zeynep Bal	032390103
Nihad Gasimzade	032390146
Rakhimzhan Abdrassulov	032390143

2025 – 2026

1. Projenin Tanımı

Bu projede, bir web tarayıcısının HTML dokümanını nasıl işlediği sadeleştirilmiş biçimde simüle edildi. Girdi olarak verilen ham HTML metni, tek geçişli bir ayrıştırıcı (HtmlParser) tarafından doğrudan karakter karakter taranarak bellekte hiyerarşik bir N-ary ağaç yapısına (HtmlNode) dönüştürüldü. Açılış/kapanış etiketi sıralaması bağlı liste tabanlı CustomStack ile takip edildi. Oluşturulan ağaç üzerinde kimlik (id) bazlı hızlı arama için CustomHashTable, genişlik öncelikli arama için ise bağlı liste tabanlı CustomQueue kullanıldı. Tüm bu veri yapıları standart kütüphanelerden yararlanılmadan sıfırdan yazıldı.

Proje; ASP.NET Core 8.0 üzerinde çalışan iki endpoint'li bir RESTful API (/api/parse ve /api/parse/search) ve saf HTML/CSS/JavaScript ile yazılmış bir arayüzden oluşmaktadır. Servis, Docker ile tek komutla ayağa kaldırılabilir şekilde yapılandırılmıştır.

2. Takım ve Görev Dağılımı

Her takım üyesi projenin belirli bir katmanını veya modülünü üstlendi. Geliştirme süreci boyunca main dalına doğrudan commit gönderilmedi; her modül ayrı bir branch üzerinde geliştirildi ve Pull Request açılarak ana dala birleştirildi.

Ad Soyad	Öğrenci No	Üstlenilen Görev
Mehmet Fatih Erduran	032490002	HtmlParser modülü içindeki ParseTagContent ve ExtractAttribute metodları — Regex tabanlı etiket adı ve öznitelik ayrıştırma. ASP.NET Core endpoint'leri (ParseController) ve CORS yapılandırması (Program.cs).
Kerem Bilgiç	032490001	HtmlParser.ParseToTree() — bağlı liste tabanlı CustomStack kullanarak N-ary ağaç inşası. HtmlNode modeli (dinamik dizi genişletme, Parent pointer, Index alanı). Void element ve script/style bloğu işleme.
Zeynep Bal	032390103	CustomHashTable implementasyonu — Polynomial Rolling Hash (Horner metodu), Separate Chaining ile çakışma çözümü. DomSearcher içinde BFS ile hash tablosu index oluşturma.
Nihad Gasimzade	032390146	CustomQueue implementasyonu — bağlı liste tabanlı. DomSearcher.Search() — id sorguları için O(1) hash tablosu, class/tag sorguları için O(N) BFS. Ağaç derinliği ve kardeş düğüm fonksiyonları.
Rakhimzhan Abdrassulov	032390143	Frontend arayüzü (032390143_dom-front.html) — collapsible N-ary ağaç görselleştirmesi, BFS tabanlı arama, Cyberpunk/Catppuccin Mocha teması, JetBrains Mono/Syne fontları. Docker yapılandırması (Dockerfile, docker-compose.yml).

3. Sistem Mimarisi

Sistem iki ana bileşenden oluşuyor: ASP.NET Core 8.0 tabanlı bir backend API ve saf JavaScript ile yazılmış bir arayüz. İkisi arasındaki iletişim HTTP/JSON üzerinden sağlanıyor, CORS ayarları tüm origin'lere izin verecek şekilde yapılandırıldı.

3.1 Veri Akışı

Parse isteği (/api/parse):

- Kullanıcıdan gelen ham HTML string'i POST /api/parse endpoint'ine iletiliyor.

- `HtmlParser.ParseToTree()`, metni karakter karakter tarayarak `CustomStack` yardımıyla N-ary ağaç oluşturuyor.
- API, kök `HtmlNode`'u JSON formatında frontend'e döndürüyor.
- Frontend, gelen JSON'ı collapsible ağaç görünümünde render ediyor.

Arama isteği (/api/parse/search):

- Frontend, arama sorgusunu ve HTML metnini `POST /api/parse/search` endpoint'ine gönderiyor.
- `DomSearcher`, ağacı BFS ile gezerek `CustomHashTable`'a id değerlerini kaydediyor.
- id sorgusu gelirse $O(1)$ hash tablosu araması yapılıyor; `class/tag` sorguları için BFS $O(N)$ kullanılıyor.
- Sonuç düğüm listesi JSON olarak frontend'e iletiliyor; eşleşen düğümler vurgulanıyor.

3.2 Modül ve Dosya Yapısı

Dosya	Sorumlu	Açıklama
backend/DomParserApi/HtmlParser.cs	Kerem Bilgiç / M.F. Erduran	Tek geçişli HTML parser, Stack tabanlı N-ary Tree inşası, void element ve script bloğu işleme
backend/DomParserApi/HtmlNode.cs	Kerem Bilgiç	Ağaç düğümü — dinamik dizi tabanlı Children, Parent pointer, Index alanı
backend/DomParserApi/CustomStack.cs	Kerem Bilgiç	Bağlı liste tabanlı <code>Stack<HtmlNode></code> (from scratch)
backend/DomParserApi/CustomHashTable.cs	Zeynep Bal	Separate Chaining hash tablosu, Horner hash fonksiyonu
backend/DomParserApi/CustomQueue.cs	Nihad Gasimzade	Bağlı liste tabanlı <code>Queue<HtmlNode></code> (from scratch)
backend/DomParserApi/Parser/DomSearcher.cs	Nihad Gasimzade / Zeynep Bal	BFS ile index oluşturma, $O(1)$ id araması, $O(N)$ class/tag BFS araması
backend/DomParserApi/Parser/HtmlParser.cs	M.F. Erduran	ParseController bağımlılığı, Regex ile tag/attribute parse
backend/DomParserApi/Controllers/ParseController.cs	M.F. Erduran	REST endpoint'leri (/api/parse ve /api/parse/search), DI entegrasyonu
backend/DomParserApi/Models/SearchRequest.cs	M.F. Erduran	Arama isteği modeli (Html + Query alanları)
backend/DomParserApi/Program.cs	R. Abdrassulov	ASP.NET Core uygulama yapılandırması, CORS, statik dosya servisi

Dosya	Sorumlu	Açıklama
032390143_dom-front.html	R. Abdrassulov	Cyberpunk temalı frontend, collapsible N-ary ağaç, BFS arama
backend/DomParserApi/Dockerfile	R. Abdrassulov	Multi-stage .NET 8 build imajı
docker-compose.yml	R. Abdrassulov	Servis tanımı, port 5175 yönlendirmesi

4. Veri Yapıları

4.1 N-ary Tree (HtmlNode)

DOM yapısını bellekte temsil eden temel veri yapısıdır. Her düğüm şu alanları barındırır: Tag (etiket adı), Id, ClassName, Text (metin düğümleri için), Index (frontend ile düğüm eşleşmesi için), Parent (JSON serializasyonunda [JsonIgnore] ile döngüsel referans engellenir) ve dinamik dizi tabanlı çocuk listesi.

Çocuk listesi manuel bir dizi (_children) ile yönetilir:

- Başlangıç kapasitesi 2.
- ChildCount == _children.Length koşulu sağlandığında dizi 2 katına genişletilir.
- AddChild() amortize $O(1)$ karmaşıklığa sahiptir; yeniden boyutlandırma anında $O(N)$.
- GetChildren() sadece dolu olan ChildCount kadar elemanı döndürür.

4.2 Stack (Yığıt) — CustomStack

Bağlı liste tabanlı implementasyon. Her StackNode, HtmlNode verisi ve bir Next referansı taşır. _top pointer'ı her zaman yığıtın tepesini gösterir.

- Push(node) — yeni StackNode oluştur, _top'a bağla. $O(1)$.
- Pop() — _top.Data'yı al, _top = _top.Next. $O(1)$.
- Peek() — _top?.Data. $O(1)$.
- IsEmpty() — _top == null kontrolü.

Eski raporda belirtilen List<T> tabanlı implementasyonun aksine, gerçek kodda (CustomStack.cs) bağlı liste kullanılmaktadır. Bu yaklaşımın avantajı: bellek önceden ayrılmaz, dizinin yeniden boyutlandırılması gerekmez.

4.3 Hash Table (Karma Tablo) — CustomHashTable

Zeynep Bal tarafından Separate Chaining yöntemiyle yazıldı. Başlangıç kapasitesi 100 kova; sabit kapasiteli bu implementasyonda Resize mekanizması bulunmuyor.

- Hash fonksiyonu: Polynomial Rolling Hash (Horner metodu) — $\text{hash} = (\text{hash} * 31 + c) \% \text{capacity}$.
- Çakışma çözümü: Her kova bir HashEntry bağlı listesi tutuyor. Yeni eleman listenin sonuna ekleniyor.
- Insert() — mevcut anahtar varsa değeri günceller, yoksa zincirin sonuna ekler. $O(1)$ ortalama.
- Get() — kovayı bulur, zincirde key eşleşmesini arar. $O(1)$ ortalama, $O(K)$ en kötü.

DomSearcher, ağacı BFS ile gezerken id değerlerini bu tabloya kaydeder. Arama anında id sorgusu geldiğinde $O(1)$ ile sonuç döndürülür.

4.4 Queue (Kuyruk) — CustomQueue

Nihad Gasimzade tarafından bağlı liste tabanlı olarak yazıldı. _head ve _tail referansları tutulur.

- Enqueue(node) — yeni QueueNode oluştur, `_tail.Next`'e bağla, `_tail`'i güncelle. $O(1)$.
- Dequeue() — `_head.Data`'yı al, `_head = _head.Next`; `_head` null olursa `_tail` da null. $O(1)$.
- IsEmpty() — `_head == null` kontrolü.

Hem `DomSearcher.BuildIndex()` hem de `DomSearcher.Search()` içinde BFS için kullanılır. Bellek dinamik olarak büyür; dizi tabanlı yaklaşıma göre yeniden boyutlandırma yoktur.

5. Algoritmalar

5.1 Tek Geçişli HTML Ayırıştırma — `HtmlParser.ParseToTree()`

`HtmlParser`, HTML metnini tek geçişte (single pass) karakter karakter okur. Ayır bir tokenizer sınıfı yoktur; etiket sınırları index tabanlı string işlemleriyle belirlenir.

- '`<`' karakteri bulunana kadar metin içeriği text düğümü olarak eklenir.
- '`>`' karakterine kadar tagContent okunur; yorum ve DOCTYPE atlanır.
- '`'`' ile başlayan tagContent kapanış etiketi → `stack.Pop()` çağrısı.
- Açılış etiketi: `ParseTagContent()` ile `HtmlNode` oluşturulur, ebeveyne eklenir, stack'e push edilir.
- Self-closing etiketler (tagContent sonu '`'` veya void element): stack'e eklenmez.
- `li`, `td`, `th`, `tr` için otomatik kapanış mantığı: yeni etiket geldiğinde çakışan ebeveyn otomatik pop edilir.
- `script/style` blokları içindeki metin, ilgili kapanış etiketine kadar tek metin düğümü olarak işlenir.

Zaman karmaşıklığı: $O(N)$ — N : HTML karakter sayısı. Uzay karmaşıklığı: $O(D)$ — D : maksimum ağaç derinliği (Stack boyutu).

5.2 BFS ile Index Oluşturma — `DomSearcher.BuildIndex()`

`DomSearcher` oluşturulurken `BuildIndex()` çağrılır. `CustomQueue` kullanılarak ağaç seviye seviye gezilir; id değeri olan her düğüm `CustomHashTable`'a kaydedilir.

- Root kuyruğa eklenir.
- Kuyruk boşalana kadar: `Dequeue()` → id varsa `_idTable.Insert()` → tüm Children'i `Enqueue()`.

Zaman: $O(N)$, Uzay: $O(W)$ — W : ağacın maksimum genişliği.

5.3 Arama — `DomSearcher.Search()`

Sorgu tipine göre farklı strateji kullanılır:

- `id="..."` formatındaki sorgular: Regex ile id çıkarılır, `_idTable.Get()` ile $O(1)$ sonuç alınır.
- `class="..."` formatındaki sorgular: BFS ile ağaç taranır; `ClassName.Contains()` ile eşleşme kontrol edilir. $O(N)$.
- Düz metin sorguları: BFS ile Tag, Id, ClassName ve Text alanlarında case-insensitive arama. $O(N)$.

Zaman: $O(1)$ id için, $O(N)$ class/tag için. Uzay: $O(W)$ BFS için.

5.4 Ağaç Derinliği ve Kardeş Düğüm

Ağaç derinliği $\max(\text{depth}(\text{child})) + 1$ formülüyle rekürsif hesaplanır; $O(N)$ zaman, $O(D)$ uzay. Kardeş düğümler `parent.Children` üzerinden erişilir; $O(K)$ — K : çocuk sayısı.

6. Zaman ve Uzay Karmaşıklığı Analizi

N: düğüm sayısı, D: ağaç derinliği, W: ağaç genişliği, K: hash zinciri / çocuk sayısı, T: token/metin sayısı.

İşlem	Ortalama	En Kötü	Uzay
HtmlParser.ParseToTree()	$O(N)$	$O(N)$	$O(D)$ — Stack
CustomStack.Push / Pop / Peek	$O(1)$	$O(1)$	$O(D)$
HtmlNode.AddChild — amortize	$O(1)$	$O(N)$ resize	—
CustomHashTable.Insert	$O(1)$	$O(K)$	$O(N)$
CustomHashTable.Get	$O(1)$	$O(K)$	$O(1)$
CustomQueue.Enqueue / Dequeue	$O(1)$	$O(1)$	$O(W)$
DomSearcher.BuildIndex — BFS	$O(N)$	$O(N)$	$O(W)$
DomSearcher.Search — id	$O(1)$	$O(K)$	$O(1)$
DomSearcher.Search — class/tag	$O(N)$	$O(N)$	$O(W)$
Ağaç Derinliği Hesaplama	$O(N)$	$O(N)$	$O(D)$
Kardeş Düğüm Bulma	$O(K)$	$O(K)$	$O(1)$

Not: K hash zinciri uzunluğunu ifade eder; Horner hash fonksiyonunun iyi dağılımı sayesinde pratikte 1'e yakın seyreder.

7. UML Diyagramları

7.1 Sınıf İlişkileri

Sınıf	Bağımlılıklar ve İlişkiler
HtmlParser	CustomStack (bağlı liste) kullanır. HtmlNode nesneleri oluşturur. ParseTagContent ve ExtractAttribute yardımcı metodları içerir.
HtmlNode	Self-referential: _children: HtmlNode[] (dinamik dizi), Parent: HtmlNode? [JsonIgnore]. Index int alanı frontend eşleştirmesi için kullanılır.
CustomStack	StackNode bağlı listesi tutar. StackNode: Data (HtmlNode) + Next (StackNode).
CustomHashTable	HashEntry bağlı listesi tutar. HashEntry: Key (string), Value (HtmlNode), Next (HashEntry).
CustomQueue	QueueNode bağlı listesi tutar (_head, _tail). QueueNode: Data (HtmlNode) + Next (QueueNode).
DomSearcher	_root: HtmlNode, _idTable: CustomHashTable. BuildIndex() BFS kullanır, Search() id için $O(1)$ / diğerleri için BFS $O(N)$.
ParseController	HtmlParser ve DomSearcher'a bağımlı. /api/parse ve /api/parse/search endpoint'lerini sunar.
SearchRequest	Html (string) ve Query (string) alanlarından oluşan istek modeli.

7.2 Sıralı Diyagram — Parse İsteği

- Kullanıcı → Frontend: HTML girer, 'Analiz Et' veya Ctrl+Enter.
- Frontend → POST /api/parse: { html: "..."}

- ParseController → HtmlParser.ParseToTree(html)
- HtmlParser → CustomStack: Push(root), her açılış etiketi için Push, kapanış için Pop.
- ParseToTree döner: HtmlNode root.
- ParseController → Frontend: JSON yanıt — HtmlNode ağacı.
- Frontend: JSON'ı okur, collapsible ağaç render eder.

7.3 Sıralı Diyagram — Arama İsteği

- Kullanıcı arama kutusuna sorgu girer.
- Frontend → POST /api/parse/search: { Html: "...", Query: "id=\"header\"" }
- ParseController → HtmlParser.ParseToTree(html) → HtmlNode root.
- ParseController → new DomSearcher(root): BuildIndex() BFS ile id'leri hash tablosuna yazar.
- DomSearcher.Search(query): id sorgusu → O(1) hash; class/tag → O(N) BFS.
- ParseController → Frontend: JSON — eşleşen HtmlNode listesi.
- Frontend: eşleşen düğümleri vurgular, ancestor düğümleri açar.

8. Versiyon Kontrolü ve Dağıtım

8.1 Git İş Akışı

Proje boyunca main dalına doğrudan commit gönderilmedi. Her üye kendi modülü için ayrı bir branch açtı, geliştirmesini tamamladıktan sonra Pull Request açarak ana dala birleştirdi.

Üye	Branch	Modül
Mehmet Fatih Erduran	fatih/tokenizer	HtmlParser (ParseTagContent, ExtractAttribute), API endpoint'leri
Kerem Bilgiç	kerem/tree-builder	HtmlParser (ParseToTree), CustomStack, HtmlNode
Zeynep Bal	zeynep/hashtable	CustomHashTable, DomSearcher.BuildIndex()
Nihad Gasimzade	nihad/tree-traversal	CustomQueue, DomSearcher.Search(), ağaç analiz fonksiyonları
Rakhimzhan Abdrassulov	rakhim/frontend	Frontend (032390143_dom-front.html), Dockerfile, docker-compose.yml

8.2 Docker Yapılandırması

Rakhimzhan Abdrassulov tarafından hazırlanan yapılandırma multi-stage build stratejisi kullanıyor. İlk aşamada .NET SDK imajıyla proje derleniyor (dotnet publish -c Release), ikinci aşamada yalnızca ASP.NET Runtime imajıyla çıktı çalıştırılıyor. Bu yaklaşım son imaj boyutunu önemli ölçüde küçültüyor.

docker-compose.yml; dom-parser servisini 5175:5175 portuna yönlendiriyor ve restart: always politikasıyla çalışıyor. Frontend HTML dosyası MapFallbackToFile ile statik dosya olarak serve ediliyor.

Projeyi çalıştırmak için proje kök dizininde:

```
docker-compose up --build
```

Uygulamaya erişim: http://localhost:5175

9. Kullanıcı Arayüzü

Arayüz, 032390143_dom-front.html dosyasında 591+ satır saf HTML/CSS/JavaScript olarak yazıldı. Proje gereksinimlerinde belirtilen iki panel yapısına uyuluyor.

Sol Panel — HTML Giriş Alanı

- Kod editörü görünümünde metin alanı, Tab tuşu girintisi destekliyor.
- Ctrl+Enter kısayolu ile ayrıştırma başlatılıyor.
- 'Analiz Et' düğmesiyle de işlem tetiklenebiliyor.

Sağ Panel — DOM Ağacı Görselleştirme

- N-ary ağaç, tarayıcı DevTools veya dosya sistemi görünümünü andıran açılır/kapanır (collapsible) yapıda gösteriliyor.
- Derinlik bazlı renklendirme her seviyeyi görsel olarak ayırt ediyor.
- Animasyonlar ağaç genişletme/daraltma geçişlerini yumuşatıyor.
- Düğüm Index değerleri frontend-backend eşleştirmesi için kullanılıyor.

Arama

- id araması: Backend'e /api/parse/search isteği gönderilir; DomSearcher O(1) hash tablosu ile sonuç döndürür.
- tag / class araması: DomSearcher BFS ile O(N) tarama yapar.
- Eşleşen düğüm vurgulanıyor; ancestor düğümler otomatik açılıyor.

Tasarım

- Catppuccin Mocha tabanlı Cyberpunk karanlık tema.
- JetBrains Mono (kod alanı) ve Syne (başlıklar) fontları.
- Scanline efekti (retro terminal görünümü) ve yumuşak CSS geçişleri.

10. Yapay Zeka Araçlarının Kullanımı ve Prompt Dökümü

Proje geliştirme sürecinde ChatGPT (GPT-4o) ve GitHub Copilot, yardımcı araç olarak kullanıldı. Temel veri yapıları ve algoritmalar tamamen ekip tarafından sıfırdan yazıldı; AI araçları yalnızca aşağıda belirtilen alanlarda destek amacıyla kullanıldı.

10.1 Kullanım Alanları ve Gerekçeler

Kullanım Alanı	Araç	Gerekçe
Test HTML veri seti üretimi	ChatGPT GPT-4o	Farklı topolojilerde (derin iç içe, düz liste, tablo ağırlıklı, metin yoğun) çok sayıda HTML örneği üretmek için
Regex pattern doğrulama	ChatGPT GPT-4o	ExtractAttribute() içindeki attribute parsing Regex'inin edge case'lerde (boşluklu değerler, tek/çift tırnak karışımı) doğru çalışıp çalışmadığını test etmek için
Docker yapılandırması	ChatGPT GPT-4o	Multi-stage Dockerfile'da .NET 8 SDK ve Runtime imaj sürümleri ile WORKDIR yapısının doğrulanması için
CSS tema geliştirme	GitHub Copilot	Cyberpunk / Catppuccin Mocha teması için CSS değişken isimlendirmesi ve renk paleti önerileri almak için

Kullanım Alanı	Araç	Gerekçe
Hata ayıklama	GitHub Copilot	Frontend collapsible ağaç render sırasında oluşan null reference hatalarının tespit edilmesine yardımcı olmak için

10.2 Örnek Prompt'lar ve Yanıtların Kullanımı

Prompt 1 — Test HTML Veri Seti Üretimi (Zeynep Bal)

Kullanım amacı: CustomHashTable ve DomSearcher'ın farklı senaryolarda doğru çalışıp çalışmadığını test etmek.

Gönderilen prompt:

```
Generate 5 different HTML snippets for testing a DOM parser. Each snippet should test a different scenario: (1) deeply nested elements up to 5 levels, (2) multiple elements sharing the same class name, (3) elements with both id and class attributes, (4) a table structure with tr/td, (5) mixed text nodes and elements. Keep each snippet under 20 lines. Use simple, valid HTML.
```

Yanıtın kullanım şekli: Üretilen 5 HTML snippet doğrudan test girdisi olarak kullanıldı. Bunların hiçbirisi kaynak koda dahil edilmedi; yalnızca manuel test sırasında parse doğruluğunu kontrol etmek için kullanıldı. Derinlik 5 olan snippet'te DomSearcher.Search() sonsuz döngüye giriyordu — bu, CustomQueue'da Enqueue sırasındaki null check eksikliğini ortaya çıkardı ve manuel olarak düzeltildi.

Prompt 2 — Regex Doğrulama (Mehmet Fatih Erduran)

Kullanım amacı: ExtractAttribute() metodunda kullanılan Regex pattern'inin farklı attribute formatlarında doğru çalışıp çalışmadığını doğrulamak.

Gönderilen prompt:

```
Does this C# regex correctly extract attribute values from HTML tag strings? Pattern: attrName + @"s*=\s*["'"]([^\s'"']*)["'"]". Test cases: id="main", class='container btn', class="btn primary", data-value="test value". Are there edge cases where it fails?
```

Yanıtın kullanım şekli: AI, boşluk içeren class değerlerinde (ör. class="btn primary") ilk boşluğa kadar doğru eşleşme yapıldığını ama tüm değer alındığını doğruladı — bu beklenen davranış. Ayrıca tek tırnak ile çift tırnak karışımının (id='x' gibi) pattern'i bozabileceğini belirtti. Bu bilgi doğrultusunda ExtractAttribute() kodu değiştirilmedi; ancak parser'ın bu edge case'i sessizce yuttuğu belgelendi.

Prompt 3 — Dockerfile Multi-Stage Build (Rakhimzhan Abdrassulov)

Kullanım amacı: .NET 8 uygulamasının production-ready, küçük boyutlu Docker imajı oluşturma için Dockerfile yapısını doğrulamak.

Gönderilen prompt:

```
Write a minimal multi-stage Dockerfile for an ASP.NET Core 8 Web API project. The build stage should use the SDK image, run 'dotnet publish -c Release', and the runtime stage should use only the aspnet runtime image. The app listens on port 5175. Make sure the final image is as small as possible.
```

Yanıtın kullanım şekli: AI'nın önerdiği Dockerfile yapısı referans olarak alındı ve projenin dizin yapısına (backend/DomParserApi) göre WORKDIR ve COPY yolları manuel olarak güncellendi. Son Dockerfile'daki içerik ekip tarafından uyarlanmış ve doğrulanmış versiyondur.

Prompt 4 — Frontend Arama Mantiğı Hata Ayıklama (Rakhimzhan Abdrassulov)

Kullanım amacı: Frontend'de collapsible ağaç render edilirken arama sonucu vurgulama fonksiyonunda yaşanan null reference hatasını tespit etmek.

Gönderilen prompt:

```
In my JavaScript DOM tree renderer, when I try to highlight a search result node by traversing ancestors, I get 'Cannot read properties of null (reading parentElement)'. The node objects come from a C# backend as JSON. What is the typical cause and how do I safely walk up a tree built from JSON where parent references are null?
```

Yanıtın kullanım şekli: AI, JSON serializasyonunda [JsonIgnore] ile işaretlenmiş Parent alanının frontend'e gelmeyeceğini ve ata düğümlere erişim için frontend'in kendi DOM element referanslarını kullanması gerektiğini açıkladı. Bu bilgi doğrultusunda ata düğüm açma mantığı, backend'in Parent alanına değil, frontend'deki HTML element hiyerarşisine (element.closest()) dayandırılarak düzeltildi.

10.3 AI Kullanımının Sınırları

Aşağıdaki bileşenler tamamen ekip tarafından yazıldı; bu aşamalarda AI kod üretimi kullanılmadı:

- CustomStack, CustomQueue, CustomHashTable — sıfırdan veri yapısı implementasyonu
- HtmlParser.ParseToTree() — stack tabanlı ağaç inşaa algoritması
- DomSearcher.BuildIndex() ve Search() — BFS/hash hibrit arama mantığı
- HtmlNode dinamik dizi genişletme (AddChild) — ArrayList mantığının manuel uygulaması

AI araçları hiçbir zaman doğrudan kaynak koda kopyalanmadı. Tüm AI çıktıları referans veya doğrulama amaçlı değerlendirildi; nihai implementasyon kararları ekip tarafından alındı.

11. Değerlendirme Kriterlerine Uyum

Kriter	Durum	Kaynak Dosya
N-ary Tree sıfırdan yazıldı mı?	✓ Evet	HtmlNode.cs — dinamik dizi + Parent pointer
Stack sıfırdan yazıldı mı?	✓ Evet	CustomStack.cs — bağlı liste tabanlı
Hash Table sıfırdan yazıldı mı?	✓ Evet	CustomHashTable.cs — Horner hash, Separate Chaining
Queue sıfırdan yazıldı mı?	✓ Evet	CustomQueue.cs — bağlı liste tabanlı
DFS implemente edildi mi?	✓ Evet	Ağaç derinliği ve subtree rekürsif analizi
BFS implemente edildi mi?	✓ Evet	DomSearcher.BuildIndex() ve Search()
getElementById O(1) mi?	✓ Evet	DomSearcher.Search() — _idTable.Get()
Sol panel (HTML editör) var mı?	✓ Evet	032390143_dom-front.html — Tab + Ctrl+Enter
Sağ panel (collapsible ağaç) var mı?	✓ Evet	032390143_dom-front.html — DevTools benzeri
id/class bazlı arama var mı?	✓ Evet	DomSearcher + /api/parse/search endpoint
Docker çalışıyor mu?	✓ Evet	Dockerfile + docker-compose.yml
README.md (mimari + çalıştırma) var mı?	✓ Evet	README.md
main'e doğrudan commit atılmadı mı?	✓ Evet	Branch + PR iş akışı

Kriter	Durum	Kaynak Dosya
Türkçe karakter içermeyen isimlendirme?	✓ Evet	Tüm sınıf/metod adları Latin karakter